

lab1

▼ 前序解释部分

- 操作系统分为用户态和内核态，在这里，完成xv6的lab，也就是在test测试样例下，观察用户态调用的某些封装了内核态函数的指令，是否可以由合适的内核态函数给出符合要求的输出。
- test测试样例本质上是用户态的函数调用，即使是用户态的函数，实际上只是对内核态一些函数的包装而已，所以真正要实现的部分，是xv6-k210中内核态的同义函数。

接下来用getcwd为例，展示一下从调用测试样例到执行到需要我们进行修改部分的流程。

- 例： `getcwd.c` 对应 `getcwd_test.py` 用来打分。调用时，用户态运行 `getcwd.c`

```
#include "stdio.h"
#include "stdlib.h"
#include "unistd.h"
#include "string.h"

/*
 * 测试通过时输出：
 * "getcwd OK."
 * 测试失败时输出：
 * "getcwd ERROR."
 */
void test_getcwd(void){
    TEST_START(__func__);
    char *cwd = NULL;
    char buf[128] = {0};
    cwd = getcwd(buf, 128);
    if(cwd != NULL) printf("getcwd: %s successfully!\n", buf);
    else printf("getcwd ERROR.\n");
    TEST_END(__func__);
}

int main(void){
```

```

    test_getcwd();
    return 0;
}

```

其中的 `getcwd()` 为 `syscall.c` 中声明的函数，这里则是用户态下声明的函数，可以看到，这里的参数为 `0: char *buf 1: size_t size` 即表明需要两个参数。但这个函数仅起到一个封装的作用，真正底层运行的是 `syscall()`。

```

char *getcwd(char *buf, size_t size){
    return syscall(SYS_getcwd, buf, size);
}

```

`syscall()` 其实是一个宏，这个宏在 `syscall.h` 中被定义，这些宏的作用包括将用户态的参数传入寄存器 a_0 - a_5 ，并执行 `ecall` 指令*。

而这里的第一个参数 `SYS_getcwd` 实际上是一个宏，也即用户态下声明好的，倘若希望调用 `getcwd()` 函数，则需要启用内核态进行操作，而执行这个操作的编码，就是 `SYS_getcwd` 这个宏声明好的id。

```

#define SYS_fremovexattr 16
#define SYS_getcwd 17
#define SYS_lookup_dcookie 18

```

`ecall` 是由 `trampoline.S`（汇编码）的 `uservec` 来直接执行，这会进行用户态的寄存器保存（保护现场）将相关内容存进了 `trapframe` 中，这时，我们发现文件夹已经切换到了xv6-k210下的kernel中了，这说明此时一旦结束了 `ecall`，就彻底陷入了内核态。

```

trampoline:
.align 4
.globl uservec
uservec:
    #
    # trap.c sets stvec to point here, so
    # traps from user space start here,
    # in supervisor mode, but with a
    # user page table.
    #
    # sscratch points to where the process's p

```

```

->trapframe is
    # mapped into user space, at TRAPFRAME.
    #

    # swap a0 and sscratch
    # so that a0 is TRAPFRAME
    csrrw a0, sscratch, a0

    # save the user registers in TRAPFRAME
    sd ra, 40(a0)
    sd sp, 48(a0)
    ...
    sd t6, 280(a0)

    # save the user a0 in p->trapframe->a0
    csrr t0, sscratch
    sd t0, 112(a0)

    # restore kernel stack pointer from p->trapframe->kernel_sp
    ld sp, 8(a0)

    # make tp hold the current hartid, from p->trapframe->kernel_hartid
    ld tp, 32(a0)

    # load the address of usertrap(), p->trapframe->kernel_trap
    ld t0, 16(a0)

    # restore kernel page table from p->trapframe->kernel_satp
    ld t1, 0(a0)
    csrw satp, t1

    # sfence.vma zero, zero
    sfence.vma

```

```

        # a0 is no longer valid, since the kernel
page
        # 也就是说，这里的a0寄存器只能用于存储内核页的指
针，而不可再用
        # table does not specially map p->tf.

        # jump to usertrap(), which does not retur
n
        # 跳转至usertrap()，可以发现，这里汇编码中并不设
计ret
        # 也就是自此以后进入了内核态的C语言层了
        jr t0

```

`ecall` 的执行结束，会从 `uservec` 直接跳转至C代码层的 `usertrap()` 函数进行后续的执行。

`usertrap()` 定义在 `trap.c` 中，会检查这次跳转至内核态的申请，是否来自于用户态，否则会导致panic，接下来进行用户上下文如进程指针的保存。下一步，根据异常类型（`trap==8` 即为用户态调用了内核态来获取某些信息）分发，首先修改 `epc`，使其返回后执行 `ecall` 的下一条指令（4字节后，因为是RISCV架构）；其次，允许中断（`intr_on()`），正式调用内核态的 `syscall()`。

```

void
usertrap(void)
{
    // printf("run in usertrap\n");
    int which_dev = 0;

    if((r_sstatus() & SSTATUS_SPP) != 0)
        panic("usertrap: not from user mode");

    // send interrupts and exceptions to kerneltrap
(),
    // since we're now in the kernel.
    w_stvec((uint64)kernelvec);

    struct proc *p = myproc();

```

```

    // save user program counter.
    p->trapframe->epc = r_sepc();

    if(r_scause() == 8){
        // system call
        if(p->killed)
            exit(-1);
        // sepc points to the ecall instruction,
        // but we want to return to the next instruction.
        p->trapframe->epc += 4;
        // an interrupt will change sstatus & c registers,
        // so don't enable until done with those registers.
        intr_on();
        syscall();
    }

    ...

    usertrapret();
}

```

- 接下来的部分将在内核态进行，上面的“用户态→内核态”切换过程大致粗略地结束了。

`syscall()` 会进行当前进程信息的提取，并检查以及分发任务，实际上，他像是从进程中抽取要执行的内核态函数id，并检查id是否可用，转到执行对应内核态函数的分拣中转站。既然涉及到一些结构体，就应该解释一下 `xv6-k210/kernel/syscall.c` 整个脚本*的一些关键定义部分了。

▼ `xv6-k210/kernel/syscall.c`

```

#include "include/sysnum.h"
#include "include/types.h"
#include "include/param.h"
#include "include/memlayout.h"
#include "include/riscv.h"

```

```

#include "include/spinlock.h"
#include "include/proc.h"
#include "include/syscall.h"
#include "include/sysinfo.h"
#include "include/kalloc.h"
#include "include/vm.h"
#include "include/string.h"
#include "include/printf.h"
#include "include/sbi.h"

// Fetch the uint64 at addr from the current process.
int fetchaddr(uint64 addr, uint64 *ip)
{
    struct proc *p = myproc();
    if (addr >= p->sz || addr + sizeof(uint64) > p->sz)
        return -1;
    // if(copyin(p->pagetable, (char *)ip, addr, sizeof(*ip)) != 0)
    if (copyin2((char *)ip, addr, sizeof(*ip)) != 0)
        return -1;
    return 0;
}

// Fetch the nul-terminated string at addr from the current process.
// Returns length of string, not including nul, or -1 for error.
int fetchstr(uint64 addr, char *buf, int max)
{
    // struct proc *p = myproc();
    // int err = copyinstr(p->pagetable, buf, addr, max);
    int err = copyinstr2(buf, addr, max);
    if (err < 0)
        return err;
}

```

```

    return strlen(buf);
}

static uint64
argraw(int n)
{
    struct proc *p = myproc();
    switch (n)
    {
        case 0:
            return p->trapframe->a0;
        case 1:
            return p->trapframe->a1;
        case 2:
            return p->trapframe->a2;
        case 3:
            return p->trapframe->a3;
        case 4:
            return p->trapframe->a4;
        case 5:
            return p->trapframe->a5;
    }
    panic("argraw");
    return -1;
}

// Fetch the nth 32-bit system call argument.
int argint(int n, int *ip)
{
    *ip = argraw(n);
    return 0;
}

// Retrieve an argument as a pointer.
// Doesn't check for legality, since
// copyin/copyout will do that.
int argaddr(int n, uint64 *ip)
{

```

```

    *ip = argraw(n);
    return 0;
}

// Fetch the nth word-sized system call argumen
t as a null-terminated string.
// Copies into buf, at most max.
// Returns string length if OK (including nul),
-1 if error.
int argstr(int n, char *buf, int max)
{
    uint64 addr;
    if (argaddr(n, &addr) < 0)
        return -1;
    return fetchstr(addr, buf, max);
}

extern uint64 sys_chdir(void);
extern uint64 sys_close(void);
...

static uint64 (*syscalls[])(void) = {
    [SYS_fork] sys_fork,
    [SYS_exit] sys_exit,
    ...
};

static char *sysnames[] = {
    [SYS_fork] "fork",
    [SYS_exit] "exit",
    ...
};

void syscall(void)
{
    int num;
    struct proc *p = myproc();

```



```

    num = p->trapframe->a7;
    if (num > 0 && num < NELEM(syscalls) && syscalls[num])
    {
        p->trapframe->a0 = syscalls[num]();
        // trace
        if ((p->tmask & (1 << num)) != 0)
        {
            printf("pid %d: %s -> %d\n", p->pid, sysnames[num], p->trapframe->a0);
        }
    }
    else
    {
        printf("pid %d %s: unknown sys call %d\n", p->pid, p->name, num);
        p->trapframe->a0 = -1;
    }
}

uint64
sys_test_proc(void)
{
    int n;
    argint(0, &n);
    printf("hello world from proc %d, hart %d, arg %d\n", myproc()->pid, r_tp(), n);
    return 0;
}

uint64
sys_sysinfo(void)
{
    uint64 addr;
    // struct proc *p = myproc();

    if (argaddr(0, &addr) < 0)
    {

```

```

        return -1;
    }

    struct sysinfo info;
    info.freemem = freemem_amount();
    info.nproc = procnum();

    // if (copyout(p->pagetable, addr, (char *)&info, sizeof(info)) < 0) {
    if (copyout2(addr, (char *)&info, sizeof(info)) < 0)
    {
        return -1;
    }

    return 0;
}

uint64 sys_shutdown(void)
{
    // printf("Shutdown hear\n");
    sbi_shutdown();
    return 0;
}

```

- `syscall.c` 的构成大概如是：
 - include
 - 封装了 `copyin2()`、`copyinstr2()` 等安全使用用户态内容的 fetch 函数
 - 解析 `trapframe` 的参数读取与转换
 - 外部 `sys_*` 函数声明（`syscall.c` 实现了部分的声明，即 `sys_shutdown()`，不过后面应该还是把它放在别的文件里实现才行）
 - 系统调用表与名字表
 - `syscall()` 分发函数
 - 后续实例调用实现&辅助函数

- 这里简要说一下大概都什么作用
 - fetch函数部分：安全地在内核与用户虚拟地址空间之间拷贝或读取数据，避免直接解引用用户指针导致内核崩溃或越权。
 - 这里的 `copyin2` 等函数，来自于头文件 `vm.h`，是安全从用户态空间做地址转换后拷贝到内核态的函数
 - `argraw` / `argint` 等：解析当前进程（`*p = myproc()`）的 `trapframe` 结构体，并将其内容转换成内核态可以直接用的变量，都使用指针来进行参数传递以及结果传递，不是直接返回结果。
 - 这里的 `trapframe` 是一个结构体，上文提到过要把 `epc` 向后移动4 Byte，`epc` 实际上就是用户态的 `pc` 指针，指着要执行的指令。后面还存了一大堆寄存器。

```
struct trapframe {
    /* 0 */ uint64 kernel_satp;    // k
    /* 8 */ uint64 kernel_sp;      // t
    /* 16 */ uint64 kernel_trap;   // u
    /* 24 */ uint64 epc;           // s
    /* 32 */ uint64 kernel_hartid; // s
    /* 40 */ uint64 ra;
    /* 48 */ uint64 sp;
    ...
    /* 112 */ uint64 a0;
    /* 120 */ uint64 a1;
    /* 128 */ uint64 a2;
    /* 136 */ uint64 a3;
    /* 144 */ uint64 a4;
    /* 152 */ uint64 a5;
    /* 160 */ uint64 a6;
    /* 168 */ uint64 a7;
    ...
}
```

```
/* 280 */ uint64 t6;
};
```

- `argraw`：一个结构体，实际上干了一个小函数的工作，将 `trapframe` 存的几个a寄存器内容直接放在 `argraw` 里。
- `argint(int n, int *ip)`：取系统调用时传递的第n个参数 (a_n 寄存器内容)，这里的 `ip` 指向一个 `argraw` 结构体，直接让它完成对于 `trapframe` 的参数取用。
- `argaddr(int n, uint64 *ip)`：同上面的逻辑，但这里的 `*ip` 是一个64位无符号整数指针，用于将 `trapframe` 中的第n参数地址存在 `*ip` 指针中传出。
- `argstr(int n, char *buf, int max)`：检查一下传入的地址（第n个word-size的位置）是否合法，将其传入 `buf` 开好的数组内，最大传 `max` 个，并返回传递了多少个char。
- 外部 `sys_*` 函数声明：告诉 `syscall()` 这些函数在外部实现过了，同时给后面的调用表提供一个函数接口。（这块就是，如果实现什么新的内核态函数，要在这里添加声明）。
- 系统调用表和名字表：
 - `static uint64 (*syscalls[])(void)`：这是一个存放函数指针的结构体，把系统调用号（宏 `SYS_fork`）与真正要调用的内核态函数建立映射。
 - `static char *sysnames[]`：名字表，一个char数组，用于将系统调用号和名字建立映射。
- `syscall()`：核心分发函数。
- `syscall()`：核心分发函数，从 `trapframe` 的 a_7 中取得系统调用号（`SYS_*`）
`p->trapframe->a0 = syscalls[num]();` 这一步则直接调用了系统调用表中的内核态函数，并将返回值放入了 a_0 寄存器中。
- 至此，我们完成了从用户态运行测试样例，到最终执行内核态函数的过程。

▼ 其他脚本解释

- `sysnum.h`：声明了系统调用号的宏（`#define SYS_getcwd 17`），如果有新的函数，要在这声明系统调用号

- `init.c`：这是用户态执行的第一个脚本，这里就包含了执行测试样例的部分，对于 `tests[]` 中的每个测试名，`fork()` 出一个子进程，子进程 `exec` 一个 `fs.img` 镜像中的可执行文件，也就是测试样例（例：`getcwd.c`）。最后依次执行完毕，执行一个 `shutdown()` 关闭系统调用。

```
char *argv[] = {0};
char *tests[] = {
    "getcwd",
    "write",
    "getpid",
    "times",
    "uname",
};
...
int main(void)
{
    int pid, wpid;

    // if(open("console", O_RDWR) < 0){
    //     mknod("console", CONSOLE, 0);
    //     open("console", O_RDWR);
    // }
    dev(O_RDWR, CONSOLE, 0);
    dup(0); // stdout
    dup(0); // stderr

    for (int i = 0; i < counts; i++)
    {
        printf("init: starting %s\n", tests[i]);
        pid = fork();
        if (pid < 0)
        {
            printf("init: fork failed\n");
            exit(1);
        }
        if (pid == 0)
        {
            // printf("IIIinit.c testing\n");
        }
    }
}
```

```

        exec(tests[i], argv);
        printf("init: exec %s failed\n", tests[i]);
        exit(1);
    }

    for (;;)
    {
        // this call to wait() returns if the shell exits,
        // or if a parentless process exits.
        wpid = wait((int *)0);
        if (wpid == pid)
        {
            // the shell exited; restart it.
            break;
        }
        else if (wpid < 0)
        {
            printf("init: wait returned an error\n");
            exit(1);
        }
        else
        {
            // it was a parentless process; do nothing.
        }
    }
}
shutdown();
return 0;
}

```

- `fs.img`：一个镜像，挂载测试样例的所有可执行文件。
- `usys.pl` (perl脚本)：生成 `usys.S`，这个汇编代码会把系统调用号load进入 a_7 寄存器，包含了 `sysnum.h` 头文件，直接将宏对应整数加载到寄存器中。如果有新的函数，必须在这里注册 `entry`，只有这样，才能为我们创建的函数生成一个函数符号，用户态（执行测试样例）在进行 `ecall` 时，前一步的 `syscall(SYS_getcwd, buf, size);` 将用户态的系统调用号传入 a_7 ，但如果用户态这一端没有 `usys.S` 中对应 `entry`，就找不到要链接哪个函数符号了。

```

print "# generated by usys.pl - do not edit\n";

print "#include \"kernel/include/sysnum.h\"\n";

# 下面这个sub entry就是后面产生.S文件的生成器，每个后面声明的
entry都会有一个对应的.S条目

sub entry {
    my $name = shift;
    print ".global $name\n";
    print "${name}:\n";
    print " li a7, SYS_${name}\n";
    print " ecall\n";
    print " ret\n";
}

entry("fork");
entry("exit");
entry("wait");
entry("pipe");
...

```

▼ getcwd

- 用户态的两个参数之一（`size`），就通过 `trampoline` 在保护现场的同时，存入了 `trapframe` 中，这样我们就可以用 `argint(1, &size)` 直接得到 `size` 这个int类型参数了
- 这里看到 `getcwd()` 有两个参数，一个是预留下来用于存储后面地址的char数组 `buf`，一个就是这个char数组的大小。如果成功getcwd，`cwd` 将不是NULL。

```

void test_getcwd(void){
    TEST_START(__func__);
    char *cwd = NULL;
    char buf[128] = {0};

```

```

        cwd = getcwd(buf, 128);
        if(cwd != NULL) printf("getcwd: %s successfully!
\n", buf);
        else printf("getcwd ERROR.\n");
        TEST_END(__func__);
    }

```

- 查找内核态对应的内核态函数 `sys_getcwd()`，这是改完之后的样子，要加入第二个参数size的判断逻辑，并修改错误情况返回的值，将其设为 `NULL`。

```

uint64
sys_getcwd(void)
{
    uint64 addr;
    int size;
    if (argaddr(0, &addr) < 0 || argint(1, &size) < 0)
        return NULL;
    // if (argaddr(0, &addr) < 0)
    //     return -1;

    struct dirent *de = myproc()->cwd;
    char path[FAT32_MAX_PATH];
    char *s = path + sizeof(path) - 1;
    int len;

    if (de->parent == NULL)
    {
        s = "/";
    }
    else
    {
        s = path + FAT32_MAX_PATH - 1;
        *s = '\0';
        while (de->parent)
        { // 这步是递归地向上寻找父目录，直到根目录为止。每找到一个父目录，就把当前目录的名字写到路径字符串的前面。
            len = strlen(de->filename);
            s -= len;

```



```

        if (s <= path) // can't reach root "/"
            return NULL;
        strncpy(s, de->filename, len);
        s--;
        if (s <= path) // can't reach root "/"
            return NULL;
        *s = '/';
        de = de->parent;
    }
}

// if (copyout(myproc()->pagetable, addr, s, strlen
(s) + 1) < 0)
    if (size < strlen(s) + 1) // check buffer size
        return NULL;
    if (copyout2(addr, s, strlen(s) + 1) < 0)
        return NULL;

    return addr;
}

```

▼ shutdown

- 在init.c的main()最后，应该调用一个shutdown()，否则不能正常通过测试点，退出系统。而这个sys_shutdown()并不是syscall.c中注册过的函数，因此需要为这个函数进行注册。这个函数的具体实现，实际上封装一下sbi_shutdown()就行了。

```

uint64
sys_shutdown(void)
{
    // printf("Shutdown hear\n");
    sbi_shutdown();
    return 0;
}

```

- 为什么不能直接在 `init.c` 中直接调用 `sbi_shutdown()`：`init.c` 是用户态的脚本存放在xv6-user文件夹下，但sbi相关函数是平台/固件接口，是内核态的函

数，相关头文件 `sbi.h` 存放在kernel文件夹下，修改这个逻辑会导致 `pid 1` `initcode: unknown sys call 8`

▼ times

- `times()` 传入一个参数，也就是 `tms` 结构体实例 `mytimes` 的地址，最后不会检查这个结构体中是否有正常的数据，但是会检查用户态调用的`times()`是否正确引起了内核态函数 `sys_times`，并取得了 ≥ 0 的 `ret`。

```
struct tms
{
    long tms_utime;
    long tms_stime;
    long tms_cutime;
    long tms_cstime;
};

struct tms mytimes;

void test_times()
{
    TEST_START(__func__);

    int test_ret = times(&mytimes);
    assert(test_ret >= 0);
    printf("mytimes success\n{tms_utime:%d, tms_stime:%d, tms_cutime:%d, tms_cstime:%d}\n",
           test_ret, mytimes.tms_utime, mytimes.tms_stime, mytimes.tms_cutime, mytimes.tms_cstime);
    TEST_END(__func__);
}
```

- `syscall()`：前面我们知道，内核态函数的返回值，在这次调用结束后，由 `syscall()` 的 `p->trapframe->a0 = syscalls[num]();` 处理后会出现在 `trapframe` 中的 `a0` 变量中，并最后恢复上下文时，存在 `a0` 寄存器里，
- `copyout2`：它把内核缓冲区的数据复制到进程的用户虚拟地址空间（由 `addr` 指定），写入的是用户内存，不会写入 `trapframe`。如果 `copyout2` 成功，用户进程在自己的地址 `addr` 上就能读到那段数据。在这个测试样例中，这个地址也就是指向 `tms` 结构体的指针。

- 也就是说如果能够让内核态函数实现正确返回0，这个测试样例就有分，甚至不用管是否给 `&mytimes` 里正确赋值。这样实测就可以通过。但是正确的逻辑是，使用 `timer.c` 里的 `ticks`，并在 `timer.h` 中声明一个跟用户态 `tms` 结构体一样的结构，用于对应地址指针 `&mytimes` 中的各部分，最后用 `copyout2`，将正确答案传回。
- 不过值得注意的是：`sys_times()` 在内核态并未被声明，因此我们需要给他从头到尾注册一下。

```
uint64
sys_times(void)
{
    // uint64 addr;
    // if (argaddr(0, &addr) < 0)
    //     return -1;

    // struct tms t;
    // acquire(&tickslock);
    // t.utime = ticks;
    // t.stime = ticks;
    // t.cutime = ticks;
    // t.cstime = ticks;
    // release(&tickslock);
    // if (copyout2(addr, (char *)&t, sizeof(t)) < 0)
    //     return -1;
    return 0;
}
```

▼ uname

- 跟times的要求一样，只要能正常引发内核态调用 `sys_uname()` 并返回0就行了。

```
struct utsname {
    char sysname[65];
    char nodename[65];
    char release[65];
    char version[65];
    char machine[65];
    char domainname[65];
};
```

```
};

struct utsname un;

void test_uname() {
    TEST_START(__func__);
    int test_ret = uname(&un);
    assert(test_ret >= 0);

    printf("Uname: %s %s %s %s %s %s\n",
        un.sysname, un.nodename, un.release, un.version, un.machine, un.domainname);

    TEST_END(__func__);
}
```

- 至于 `utsname` 的内容，自己写合适的就好。

```
uint64
sys_uname(void)
{
    uint64 addr;
    if (argaddr(0, &addr) < 0)
        return -1;
    struct utsname un;
    strncpy(un.sysname, "Miss", sizeof(un.sysname));
    strncpy(un.nodename, "DSY", sizeof(un.nodename));
    strncpy(un.release, ",", sizeof(un.release));
    strncpy(un.version, "I", sizeof(un.version));
    strncpy(un.machine, "love", sizeof(un.machine));
    strncpy(un.domainname, "you!", sizeof(un.domainname));
    if (copyout2(addr, (char *)&un, sizeof(un)) < 0)
        return -1;
    return 0;
}
```